



COM 341: Operating Systems

American University of Central Asia
Software Engineering Department

1 Course Description

This course is an introduction to the concepts and mechanisms behind modern operating systems. It covers the interaction between kernel and user space, process management, scheduling, memory management, file systems, and the OS-specific APIs used to tune server software. Students work in the Unix environment and take on system programming in C and assembly, at both the high and the low level. Labs and projects include building Unix utilities, programming against system calls, exploring the Linux kernel, and writing tools for FUSE-based file systems. The final project brings that work together into a small Unix-like operating system. Students leave with the theoretical grounding and the practical skills to work with modern operating systems as software engineers.

2 Course Details

Course Materials

<https://github.com/auca/com.341>

Course Code

COM 341

Course ID

3325

Prerequisites

COM 229, Data Structures
COM 123, Principles of Computing Systems

Prerequisites for

The list below depends on the student's year of admission.

COM 416.1, Computer Networks
COM 451, Parallel Programming

Credits

6

Time and Place

Lecture: Monday 12:45–14:00, Room 410

Lab: Wednesday 12:45–14:00, Room 410

3 Contact Information

Professor

Dmitrii Toksaitov

toksaitov_d@auca.kg

TA

Ruslan Albaev

ar12404@auca.kg

Office

AUCA, Room 315

Office Hours

Office hours are held by appointment, in person or remotely, during business hours (Monday–Friday). Please contact your professor or TA to arrange a meeting.

4 Topics

- Weeks 1–2: Introduction, History, Overview of OS Concepts, Terminals (6 hours)
- Weeks 3–5: System Calls (9 hours)
- Weeks 6–8: Processes, Threads, Scheduling (9 hours)
- Weeks 9–11: Virtual Memory (9 hours)
- Weeks 12–14: File Systems (9 hours)
- Weeks 15–16: Networking (6 hours)

5 Learning Outcomes

By the end of this course, students will:

1. Acquire programming skills to write and test system programs:

- Modify or develop system software such as standard Unix utilities, a command-line shell, and high-performance server software to gain practical experience with system programming

- Gain hands-on experience with kernel programming by making targeted modifications to the Linux kernel
2. **Evaluate and use appropriate tools and techniques for system software development:**
 - Use a standard Unix development environment, including editors, compilers, debuggers, and build systems
 - Work directly with low-level operating system (OS) APIs to build and optimize applications that interface with the kernel
 - Apply version control and collaborative development practices
 3. **Design, plan, and critically evaluate system software solutions to problems:**
 - Model the fundamental interactions between user-space applications and the OS kernel via the system call interface
 - Design applications that use core OS abstractions, including processes, threads, scheduling, and virtual memory
 - Analyze system requirements to design solutions for file management
 4. **Evaluate low-level systems in terms of quality attributes and trade-offs:**
 - Analyze the performance implications of different CPU scheduling algorithms and virtual memory management techniques
 - Optimize server software by applying knowledge of OS-level APIs and their interaction with hardware resources
 - Evaluate the trade-offs between different file system designs and their impact on data persistence and access patterns
 5. **Reflect on and reason about information-handling problems:**
 - Reason about the role of the operating system in managing shared resources, process isolation, and network communication
 - Critically examine techniques for managing concurrency using processes and threads
 - Analyze the principles of data persistence and abstraction as implemented in modern file systems
 6. **Understand and apply ethical principles and professional standards:**
 - Recognize the ethical implications of system software design choices, particularly in terms of security, privacy, and resource management
 - Adhere to professional standards in collaborative software development, including code quality, documentation, and version control practices
 - Reflect on the societal impact of operating systems and system software in enabling modern computing environments

6 Assignments

6.1 Moodle/e-Course Checkpoints

Students must maintain instructor-provided private GitHub repositories for their assignments. They must periodically commit and push the required number of project solutions, as directed by the course staff. Instructors or teaching assistants will review the work either during lab sessions (on-site) or after the submission deadline (off-site) and award points based on completed assignments.

6.2 Labs and Projects

Throughout the course, students complete several laboratory tasks in which they examine standard Unix utilities and then implement them in C on their own. The C versions should use only direct system calls, staying as close to the kernel as possible. Through these tasks, students explore the interfaces between the OS kernel and user-space applications. Completing these labs also prepares students for the practical assignments during the midterm and final examination sessions.

Students are required to complete four projects during the course: studying a toy shell, modifying the Linux kernel, writing utilities for a basic file system driver, and enhancing a web server using custom low-level OS APIs.

7 Course Materials, Recordings, and Screencasts

All course materials are available on GitHub at <https://github.com/auca/com.341>. Using the course repository will help students become familiar with the Git version control system and GitHub.

We aim to record each class and upload the recording to YouTube as time permits, but recordings are not guaranteed. If you need a recording right away, consider recording the session on your own computer. Links to YouTube recordings can be found in the course repository at <https://github.com/auca/com.341>.

While recordings provide flexibility, they are not a substitute for attending classes. Active participation is crucial for success in this course. Accumulating three or more unexcused absences may lead to an *X* grade. If overall class attendance is poor, the instructor reserves the right to discontinue class recordings.

Access lecture screencasts remotely via Zoom at <http://com-341-zoom.auca.space>. When joining a Zoom session, identify yourself with your properly capitalized first and last names in the Latin alphabet. Your lab instructor may also use Zoom or other tools for remote work and set additional etiquette rules. Ask your instructor for details. Install and configure the required tools on your computer during the first week of the semester so that you can use your camera and microphone and share your screen.

To attend lectures or labs remotely, you must request permission from the instructor in advance. Please send a brief email explaining why you need to attend remotely (e.g., illness, personal reasons) and provide appropriate documentation to support your request, either immediately or as a follow-up in the same email thread. If documentation is not provided within a reasonable time, your absence will be recorded as unexcused, and you may receive a grade of *X* if you accumulate

three or more unexcused absences. Because we receive many such requests, do not expect a reply to your email. You must still submit all required documentation as soon as possible in the same thread. AUAF students who are outside the country and can only participate online do not need to request this permission.

8 Software

Students are advised to install the following software on their computers at the start of the course. Additional installations may be required later.

- Git with SSH: <https://git-scm.com/downloads>

The compilers, assemblers, and debuggers are available on the remote course server.

9 Hardware

There are no specific hardware requirements: you can use the lab machines, where we guarantee that all course tasks can be completed. If you use your own computer, we recommend ample memory, fast disk I/O, and a CPU that supports modern virtualization technologies.

10 Reading

1. Operating System Concepts, Ninth Edition, by Abraham Silberschatz, Peter B. Galvin, and Greg Gagne (AUCA Library Call Number: QA76.76.O63 S5583 2013, ISBN: 978-1118063330)

10.1 Supplemental Reading

1. Understanding the Linux Kernel, Third Edition, by Daniel P. Bovet and Marco Cesati (AUCA Library Call Number: QA76.76.O63 B683 2006, ISBN: 978-0596005658)
2. Linux Kernel Development, Third Edition, by Robert Love (ISBN: 978-0672329463)
3. Windows Internals, Part 1, Sixth Edition, by Mark E. Russinovich, David A. Solomon, and Alex Ionescu (AUCA Library Call Number: QA76.76.W56 R885 2012, ISBN: 978-0735648739)
4. Windows Internals, Part 2, Sixth Edition, by Mark E. Russinovich, David A. Solomon, and Alex Ionescu (AUCA Library Call Number: QA76.76.W56 R885 2012, ISBN: 978-0735665873)
5. Mac OS X and iOS Internals: To the Apple's Core, by Jonathan Levin (AUCA Library Call Number: QA76.774.M33 L48 2013, ISBN: 978-1118057650)
6. Mac OS X Internals: A Systems Approach, by Amit Singh (AUCA Library Call Number: QA76.76.O63 S564 2007, ISBN: 978-0321278548)

11 Grading

The preliminary distribution of points is outlined below. This distribution may change during the course if tasks are canceled, merged, or made optional (for bonus points). Such changes usually result from software issues, online service outages, or class cancellations due to events outside our control.

Some learning management systems (LMSs), such as Moodle, may not calculate final scores and grades correctly until all tasks have been published in the system and properly weighted. Do NOT assume that your grade is accurate until all tasks are in the system and your instructor has asked you to review your scores and grades.

Attendance points do not count toward your final grade. They are used only to help us determine whether to assign *X* grades in the middle of the semester, and they are removed from the totals in Moodle at the end of the semester.

11.1 Moodle/e-Course Checkpoints

Your instructor will periodically announce reviews of your work. The maximum number of points available for each category of work is listed below.

- Project #1 (10%)
- Project #2 (10%)
- Project #3 (10%)
- Project #4 (10%)

11.2 Exams

You will take the midterm examination in the middle of the course and the final examination at the end. Both carry substantial weight in your grade.

- Midterm Exam (25%)
- Final Exam (35%)

11.3 Totals

The course total of 100% consists of the Moodle/e-Course Checkpoints (40%) and the two exams (60%).

11.4 Scale

- [94%–100] %: A
- [90%–94) %: A-
- [87%–90) %: B+
- [83%–87) %: B
- [80%–83) %: B-

- [77%–80)%: C+
- [73%–77)%: C
- [70%–73)%: C-
- [67%–70)%: D+
- [63%–67)%: D
- [60%–63)%: D-
- Less than 60%: F

Requests for a higher grade because a score falls marginally short of a threshold will be ignored. A score of 93.99 is an A-, NOT an A. Requests for extra assignments to earn additional points will also be disregarded.

12 Rules

In addition to the rules in this syllabus, students are required to follow the Code of Conduct of the American University of Central Asia.

12.1 Participation

Active participation in class may be rewarded with extra points at the instructor's discretion. Poor performance in class may result in points being deducted from the final grade.

Instructors may conduct unannounced checks during class, so students **MUST** be prepared for every session to avoid losing points. Students who miss a class with graded work without a valid reason will also lose points, unless the absence is due to force majeure. To avoid losing points, inform the instructors of the reason for an absence in advance.

12.2 Questions

A question from one student is often relevant to others as well. Students are therefore encouraged to post questions publicly on the discussion board of the course's LMS, such as the AUCA e-Course System, so that all students can benefit from the discussion.

Students must not post the complete source code for any task on the LMS discussion board. Any such public post results in a grade of zero for that assignment. Students should also avoid vague or overly general questions such as "Why doesn't my code work?" and are expected to analyze and debug their code carefully before asking for help.

12.3 Late Policy

Late submissions and late exams are not accepted. Exceptions may be granted at the professor's discretion only in cases of force majeure. If you fall ill, face serious personal difficulties, or have technical problems with your computer or internet connection, you must notify the instructors at least 24 hours before the deadline.

Otherwise, no extension will be granted. Requests made on the day of the deadline will be treated as procrastination and will not be considered. No student is granted more than one emergency extension during the course.

During the six hours before any assignment deadline, the instructors enter silent mode: they do not answer assignment-related questions or consider requests for office hours. At all other times, the instructors make every effort to answer questions and provide help, whether via Zoom or in person during office hours.

12.4 Exam and Task Submission Ceremonies

Students must follow all exam and task submission protocols, which means strictly adhering to every rule the instructors specify, whether in writing or verbally. Failure to do so results in a loss of points.

Throughout your career, you will work with supporting documents such as contracts and timesheets. Beyond programming, you will also use project management systems, application lifecycle management tools, version control, and continuous integration and deployment pipelines. You will manage complex configurations, collaborate with colleagues by following specific methodologies, and share code through pull requests or other means. In all these processes, proper etiquette and attention to detail are essential.

Our courses aim to help you build these skills. We deduct points for not following submission rules or protocols, and we may refuse to accept an exam defense or a submitted task altogether. We also award zero points for missed task deadlines and for violations of strict exam timing rules, such as arriving late or failing to submit exam documents on time.

12.5 Administrative Drop

Instructors reserve the right to drop a student from the course for nonattendance. If a student misses three or more classes without an acceptable excuse, the faculty may assign an *X* grade and remove the student from the course.

An absence can be excused only if you complete both of the steps below and we accept your explanation.

First, write to the instructor of the class you will miss, your lecture instructor for a lecture or your lab instructor for a lab. Send a short message giving the reason, and send it BEFORE the class. This deadline is strict: a message that arrives after the class has taken place will not be processed, and the absence will be counted as unexcused.

Second, once your health or your situation allows, you MUST upload documents supporting your explanation to the same email thread. These can be medical records, a letter from your parents, or documents from an organization or an official body, as long as they confirm that the situation was real and that it prevented you from attending. Upload them as soon as you can, since we review the threads before *X* grades are decided and cannot take later material into account.

If the situation was genuinely serious and your documents are valid, the absence may be excused. We decide each case on its own merits, and we may ask you or your guardians to contact us or to meet with us at AUCA so that the documents can be verified. We may also decline to excuse an absence if the circumstances do not warrant it, or if the documents raise doubts.

Do not expect a reply to these threads. We receive too many messages to answer them or even confirm that they arrived, so you will usually hear from us only if we need something more from you. Every thread is still collected and read carefully, so silence does not mean your message was lost or ignored. For the same reason, we do not accept appeals or negotiations about these decisions: handling them would take time away from our core work.

12.6 Incomplete Grade

Consistent with the late exam policy, a grade of *I* (Incomplete) is granted only under highly exceptional circumstances. Students must request an *I* grade well in advance of the final exam. Requests made the day before the final exam or later will not be considered.

12.7 Academic Honesty

Plagiarism is the act of copying or appropriating someone else's words or ideas and presenting them as one's own. In this course, plagiarism can involve any kind of work, including program code, comments, software documentation, design specifications, requirements documents, project reports, and technical analyses.

Examples of plagiarism in a software engineering course include:

- Submitting code written by others or generated by AI as your own
- Modifying existing code or designs (e.g., changing variable names or restructuring code) and claiming originality
- Incorporating code snippets, sentences, design patterns, or intellectual content from any source without citation
- Using algorithms, patterns, or architectural designs without proper acknowledgment
- Using graphics, datasets, audio, video, or other materials from external works without attribution
- Purchasing or otherwise acquiring code, software, or project content and presenting it as original work

Plagiarism is unethical and undermines the educational process. The consequences are as follows:

- First instance: A grade of zero for the plagiarized work and a report filed with the Registrar's Office
- Second instance: A failing grade (*F*) for the entire course

The student who plagiarizes and the student whose work was copied face the same consequences because every student is responsible for safeguarding their assignments, code, and related materials against unauthorized use. This policy reflects

professional expectations in software engineering, where failing to protect intellectual property can lead to dismissal, reputational damage, compromised customer and user safety, and even lawsuits.

All work in this course must be completed individually. Group work, collaboration with peers, and seeking help from other students are strictly prohibited. Relying on classmates often spreads misunderstandings and increases the risk of academic dishonesty. Students may seek help only from instructors, teaching assistants (TAs), and WARC tutors, so that guidance comes from qualified sources and each student develops their own skills and understanding.

The use of artificial intelligence, including generative AI tools, to complete any assignment, project, or exam, whether off-site or on-site, is strictly prohibited. A student suspected of using AI assistance will be required to perform a similar task and answer related questions in a supervised setting with the course instructors. Failure to complete this task satisfactorily or to answer the questions convincingly will lead to the conclusion that the work was AI-generated, and the Academic Honesty policies above will be enforced, including possible disciplinary action. For some tasks, the instructor may permit the use of AI tools. Such permission is valid only when it is stated explicitly in the written task requirements.

This course also follows all university-wide AUCA policies on academic honesty. Where university regulations are stricter than the rules stated here, the stricter rules apply.

12.8 Use of AI by the Course Staff

The instructors and teaching assistants for this course use artificial intelligence tools in their work. The instructors design the course, develop its content, plan instruction and assessment, and make all academic decisions themselves. They use AI for routine tasks: organizing and formatting materials, proofreading, keeping course documents consistent, and finding possible errors or inconsistencies in grading for their own review.

A person checks every grade before it is recorded. Practice work, including labs and projects, may be graded automatically or prechecked by AI, but a teaching assistant or the instructor verifies every automated grading result before it counts toward the course grade. Midterm and final examinations are graded by the instructor alone. AI may be used to precheck exam submissions, but neither AI nor a teaching assistant assigns an exam grade: the instructor reviews each submission and makes each grading decision personally, one by one, before the result is recorded.

The rules for student use of AI are set out in the Academic Honesty section.

13 Access and Support Services

We are committed to providing an inclusive learning environment that supports the needs of all students. If you have a disability or need specific accommodations to participate fully, contact the instructors as early as possible. We will work with you to make appropriate adjustments.

For guidance on time management, presentation skills, writing, or study strategies beyond the scope of software engineering, contact the AUCA Advising Office, which offers practical workshops and peer advising to help you with academic chal-

lenges.

If you would like help from a student tutor, you can schedule an appointment through the AUCA Writing and Academic Resource Center (WARC). Tutoring is available in person and online, and additional resources are listed on the WARC webpage. Note, however, that team or group work with other students is not permitted on assignments in this course.

If you feel stressed or overwhelmed or are struggling with emotions, relationships, or other mental health concerns, we encourage you to seek support from the AUCA Counseling Service, which provides free, confidential professional help to students.